Hvordan visualisere hvordan gravitasjonskraft fra ulike himmellegemer i vårt solsystem påvirker objekter?

Kevin Ngo

9. mai 2025

Innholdsfortegnelse

1 Om prosjektet og hensikt **2**

2 Teori **2**
2.1 Krefter og akselerasjon i gravitasjonsfelt 2
2.2 Unnslipningsfarten i gravitasjonsfelt 3
2.3 Sirkelbevegelse i gravitasjonsfelt 5
2.4 Elliptiske baner 6

3 Programmering i pygame **8**
3.1 Bruk av programmet 9
3.2 Fysikken bak programmet 9
3.3 Vinkel 10

4 Sjekke programmet og utforskning **11**
4.1 Sjekke 11
4.2 Utforskning 12

5 Feilkilder **12**

6 Konklusjon **12**

7 Python kode **13**

8 Kilder **23**

1 Om prosjektet og hensikt

I dette prosjektet skal jeg se på hvordan bevegelse nærme ulike himmellegemer i vårt solsystem fungerer. Jeg skal blant annet se på unnslipningsfarten til ulike himmellegemer og den minste farten nødvendig for å gå i bane rundt en himmellegeme. Jeg skal se både på sirkelformede baner og elliptiske baner. Fremgangsmåten min er en blanding av å bruke matematikk og fysikk samtidig som jeg benytter meg av programmering i python. Det overordnet målet er egentlig å utforske og se på dette gjennom en visualsering. Jeg har brukt python med pygame til å lage en todimensjonal simulasjon der brukeren kan prøve å skyte prosjektiler nærme deres valgte himmellegeme og se hvordan de beveger seg. Programmet oppgir noen nyttige verdier som farten prosjektilet skal ha, hvor lang tid det tar for en prosjektil å falle ned hvis det gjør det, og vinkelen til prosjektilet.

2 Teori

Før jeg forklarer teori som jeg har brukt i prosjektet skal jeg forklare litt av antagelsene og begrensningene jeg har satt. Først og fremst ignorerer jeg eventuell luftmotstand som kan forekomme. I tillegg ser jeg alltid bare på en himmellegeme om gangen og hvordan den påvirker ulike prosjektiler. Jeg ignorerer altså muligheten om at andre himmellegemer kan ha noe påvirkning om prosjektilene går for langt vekke fra den himmellegemen man ser på.

2.1 Krefter og akselerasjon i gravitasjonsfelt

Tiltrekningskraften mellom to gjenstander med masse m_1 og m_2 er gitt ved:

$$G = \gamma \frac{m_1 m_2}{r^2} \tag{1a}$$

Her er γ gravitasjonskonstanten med verdien $6.67 * 10^{-11} Nm^2/kg^2$ For å få akselerasjonen deres kan vi bruke det at summen av kreftene er ma er m er massen til objektet. Vi får da at:

$$a_1 = \gamma \frac{m_2}{r^2} \tag{1b}$$

$$a_2 = \gamma \frac{m_1}{r^2} \tag{1c}$$

Disse gir akselerasjonen som hver av gjenstandene vil få. Tenker man i en tilfelle der m_1 er massen et av himmellegemene i vårt solsystem, mens m_2 er massen til f.eks en sattelitt eller liknende vil akselerasjonen som himmellegemen møter være ubetydelig. Dermed er det kun a_2 som vi er interessert i. a_2 vil selvfølgelig endre størrelse etter hvert som gjenstanden enten beveger seg nærmere eller vekke fra himmellegemen. Ved bruk av programmering kan vi kjøre en løkke som da tar hensyn til denne endringen.

2.2 Unnslipningsfarten i gravitasjonsfelt

Her bruker jeg m_1 som massen til en himmellegeme og m_2 som massen til en tilfeldig gjenstand. Den totale mekaniske energien til gjenstanden med masse m_2 er gitt ved:

$$E_2 = E_k + E_p = \frac{1}{2}mv^2 - \gamma\frac{m_1 m_2}{r}$$

Når den totale mekaniske energien er lik null vil gjenstanden unslippe himmellegemen sitt gravitasjonsfelt. Vi kan bruke denne informasjonen til å regne ut hvilken fart som er nødvendig for at dette skjer.

$$E_k + E_p = 0$$

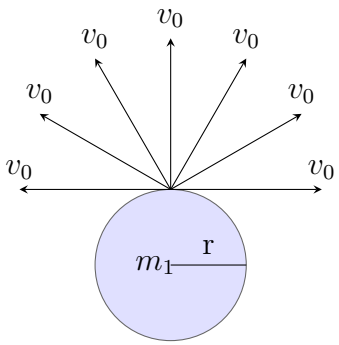
$$\frac{1}{2}mv_0^2 = \gamma\frac{m_1 m_2}{r}$$

$$v_0 = \sqrt{\frac{2\gamma m_1}{r}}$$

Her er v_0 startfarten til gjenstanden og r er radiusen til himmellegemen. Merk at m_1 referer til massen til himmellegemet, ikke selve gjenstanden. Formelen for farten nødvendig er altså uavhengig av massen til selve gjenstanden som vi ser for oss blit skutt opp fra overflaten. Dette gjør det mulig å lage en tabell for unnslipningsfarten til ulike himmellegemer basert på deres masse og radius (γ er konstant).

Himmellegeme	**Masse (10^{24} kg)**	**Radius (km)**	**Unnslipningsfart (km/s)**
Merkur	0.33	2 440	4.25
Venus	4.87	6 052	10.36
Jorden	5.97	6 371	11.19
Månen	0.073	1 737	2.38
Mars	0.642	3 390	5.03
Jupiter	1 898	69 911	59.5
Saturn	568	58 232	35.5
Uranus	86.8	25 362	21.3
Neptun	102	24 622	23.5
Pluto	0.0146	1 188	1.21

Tabell 1: Masse, radius og unnslipningsfart for ulike himmellegemer i solsystemet.



Figur 1: Illustrasjon av startsfart i forskjellige vinkler

Figur 1 som man kan se ovenfor viser eksempelet med gjenstanden som skytes opp fra overflaten av et himmellegeme. Siden unnslipningsfarten regnes ved å kun se på energi vil gjenstanden også unslippe himmellegemet om den skytes i ulike vinkler. Gravitasjonskraften er en avstandskraft. Selv om den minker veldig raskt etter hvert som sentrumsavstanden mellom himmellegemer og gjenstanden r øker vil den fremdeles virke. Derfor bør vi kunne forvente at gjenstanden alltid vil alltid ha en positiv fart (fart som peker vekk fra himmellegemet), men at det vil bli nærmere og nærmere null. Ved

bruk av dette kan jeg f.eks. sjekke at pythonprogrammet mitt funker som forventet.

2.3 Sirkelbevegelse i gravitasjonsfelt

En måte en gjenstand kan gå i bane rundt en himmellegeme er ved å gå i en sirkelbane. Det kjennetegnes ved at avstanden mellom gjenstanden og himmellegemet samt banefarten som gjenstanden er i holder seg konstant. Ved å bruke formler for gravitasjonskraften sammen med formler for sirkelbevegelse kan vi utlede den nødvendige sirklingsfarten for ulike høyder. Vi har at:

$$G = \gamma \frac{m_1 m_2}{r^2}$$

Her er igjen m_1 massen til himmellegemet. Bruker vi formelen for kraften i en sirkelbevegelse får vi at:

$$m\frac{v^2}{r} = \gamma \frac{m_1 m_2}{r^2}$$

Da får vi et uttrykk for farten:

$$v = \sqrt{\frac{\gamma m_1}{r}}$$

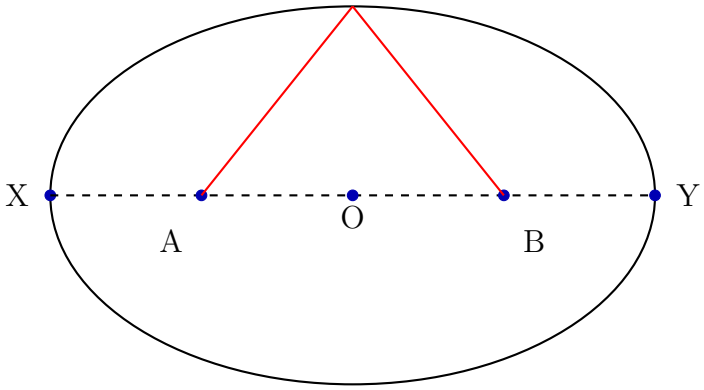
Siden r er sentrumsavstanden mellom gjenstanden og himmellegemet må man altså også ta høyden av gjenstandet over overflaten i betrakning. Desto høyere den er plassert over overflaten av himmellegemet, jo mindre fart er nødvendig for å gå i en sirkelbane rundt himmellegemet. Merk at man i praksis kan ignorere størrelsen på gjenstanden siden det som oftest er insignifikant i forhold til himmellegemet. Jeg bruker at en gjenstand har en teoretisk høyde på 0 meter over overflaten til himmellegemene. Da er det kun himmellegemene sin egen radius som er relevant. Vi kan da regne ut den nødvendige farten for å gå i denne sirkelbanen.

Himmellegeme	**Masse (10^{24} kg)**	**Radius (km)**	**Sirklingshastighet (km/s)**
Merkur	0.33	2,440	3.00
Venus	4.87	6,052	7.33
Jorden	5.97	6,371	7.91
Månen	0.073	1,737	1.67
Mars	0.642	3,390	3.56
Jupiter	1,898	69,911	42.57
Saturn	568	58,232	25.52
Uranus	86.8	25,362	15.11
Neptun	102	24,622	16.63
Pluto	0.0146	1,188	0.91

Tabell 2: Masse, radius og nødvendige sirklingsfart for himmellegemer i solsystemet.

2.4 Elliptiske baner

De sirkelformede banene vi har sett på er egentlig en spesialtilfelle av elliptiske baner. Det beskrives ofte som et flattrykt sirkel. Selve definisjonen på en ellipse er at den summen av avstanden mellom to såkalte brennpunkter og et vilkårlig punkt i ellipsen er konstant.



Figur 2: Konstruksjon av ellipse

Ovenfor i figur to kan man se hvordan man kan konstruere en ellipse. Brennpunktene A og B kan f.eks. henges i en strammet snor som har en

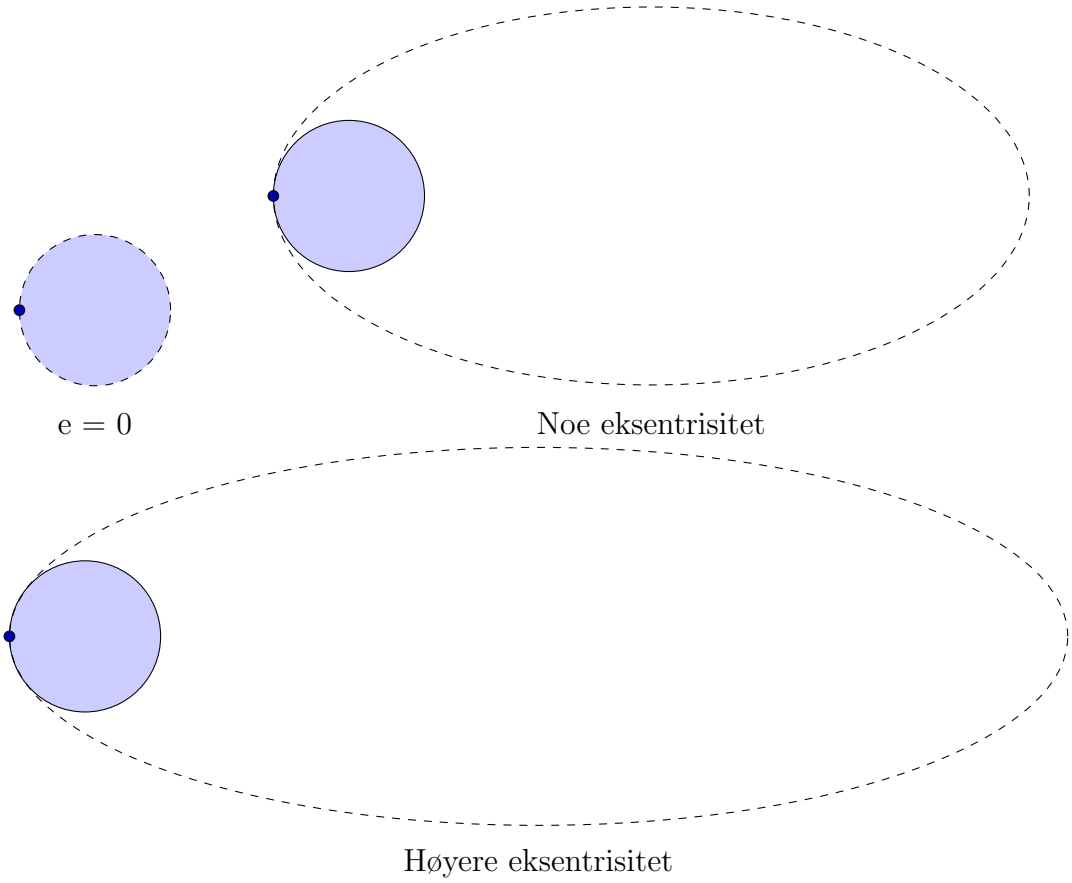
lengde som gjør at den akkurat treffer et punkt i ellipsen slik man kan se i figuren. Deretter kan man føre en blyant slik at snoren holdes stram hele tiden, og på den måten tegne en perfekt ellipse.

Eksentrisiteten til en ellipse er et mål på hvor flatklemtellipsen er. Den defineres som forholdet:

$$e = \frac{c}{a}$$

Avstanden a er lengden fra sentrum O til ytterpunktet X eller Y på den store aksen. Dette er altså halve lengden av den lange aksen til ellipsen. Avstanden c er lengden fra sentrum O til ett av brennpunktene A eller B. Eksentrisiteten e beskriver dermed hvor langt brennpunktene er trukket bort fra sentrum i forhold til hvor lang ellipsen er. Jo nærmere brennpunktene er sentrum (mindre c), desto mer ligner banen en sirkel. Nedenfor i figur 3 kan man se hvordan ulike elliptiske baner kan se ut. Vi kan ha at $e = 0$ slik at det blir en sikelformet bane, eller vi kan ha ulike mengder eksentrisitet og gjøre banen mer flattrykt.

Ved å tenke på bevaring av mekanisk energi kan vi komme fram til noe interessant om ellipseformede baner. I en ellipseformede bane vil avstanden fra et punkt i banen og sirkelen variere. Når en gjenstand i en elliptisk bane er langt vekke fra himmellegemet vil det ha en større verdi for potensiell energi enn en som er nærmere. Det vil nødvendigvis bety at gjenstanden har høyest kinetisk energi, og dermed fart, når den er nærmest himmellegemet som mulig i banen. Igjen er dette noe jeg kan senere sjekke i programmet mitt.



Figur 3: Forskjellig verdi av eksentrisitet på banene

3 Programmering i pygame

I prosjektet så har jeg brukt biblioteket pygame for å simulere en prosjektil som skytes nærme ulike himmellegemer i vårt solsystem. Brukeren trykker først en gang, for å bestemme start posisjon. Deretter må man velge en annen sted som bestemmer retningen og farten. For å gjøre det litt mer brukervennlig la jeg til funksjoner som det å zoome inn og ut, jeg la til en funksjon slik at man kan se vinkelen og hastigheten til den prosjektilen man skal til å lage, og i tillegg er det skrevet hvor lang tid det har kjørt. En av de siste funksjonene jeg la til er en slags forhåndsvisning på banen som prosjektilet kommer til å få. Man kan altså se på hvordan prosjektilen

kommer til å bevege seg før den faktisk har blitt skutt. Inpsirasjonen min for dette programmet er i stor grad eksempel 9 i kapittel 3 i Ergo fysikk 2, samt en youtubevideo som jeg har en lenke av i kildelisten. Programmet mitt er på en måte en videreutvikling av programmet i videoen, men jeg har tatt en litt annerledes tilnærming enn det han har.

3.1 Bruk av programmet

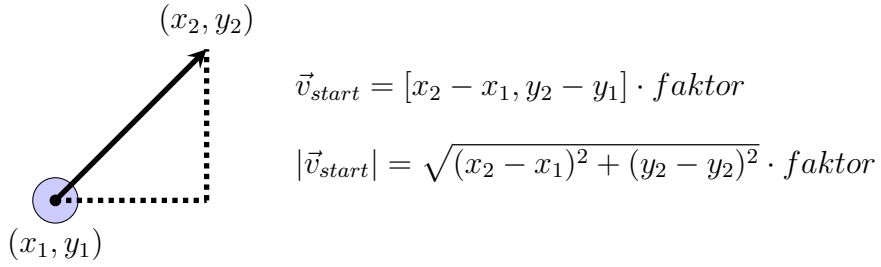
Verdiene øverst viser verdier som kan være relevant å se på. Real time forteller hvor lang tid den siste prosjektilen skutt har vært i luften i vår tid (altså et sekund er et sekund), mens sim time forteller det i forhold til hvor lang tid den må ha vært slik at det skal stemme. Resten tror jeg er nokså selvforklarende.

Noen funksjoner i programmet:

- + og - for å enten zoome inn eller ut.
- Pil til venstre og pik til høyre til å velge mellom ulike planeter.
- Trykk på d for å slette alle prosjektiler. Kan være nyttig f.eks. hvis man har for mange prosjektiler siden det kan sakte ned programmet.

3.2 Fysikken bak programmet

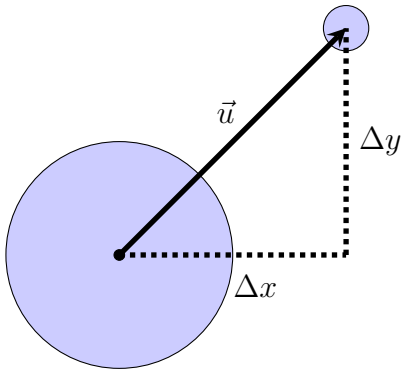
Først må prosjektilet få en startposisjon (x_1, y_1). Dette bestemmes med første museklikk. Andre museklikk i posisjon (x_2, y_2) gir retningen samt farten avhengig av distansen mellom de to punktene og en valgt skaleringsfaktor. Denne skalerings er for det første for å konvertere pikselverdier om til km og deretter velge hvor raskt man har lyst prosjektilet skal kunne bli skutt.



Figur 4: Beregning av startfarten for prosjektilen i (x_1, y_1)

Når prosjektilet først er blitt skutt må vi begynne å ta kreftene som virker i betrakning. Mer spesifikt må man tenke på tiltrekningskraften som virker på prosjektilet og hvordan det påvirker farten, og deretter posisjonen. I dette tilfellet er det nødvendig å bruke små tidssteg der man oppdaterer først akselerasjon, så fart så posisjonen. Dette er siden selve kraften og retningen på kraften endrer seg hele tiden. I mitt tilfelle brukte jeg i likhet med eksempelet i boken en Δt på 10 sekunder.

Akselerasjonen a_2 for prosjektilet klarer man å regne på - formelen for det har jeg vist tidligere. Man må merke at den formelen gir akselerasjon i absoluttverdi. Vi trenger å få retningen om vi har lyst å endre fartsvektoren riktig. Måten jeg har løst dette i programmet er ved å fått retningsvektoren fra sentrum i himmellegemet til prosjektilet. Dette kan man få ved å ta vektoren mellom de og dele det på selve lengden av den samme vektoren.



Figur 5: Figur som viser himmelegemet samt prosjektilet

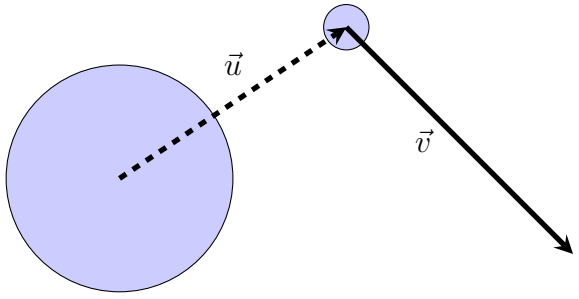
$$\vec{r}_n = \frac{\vec{u}}{|\vec{u}|} = \frac{[\Delta x, \Delta y]}{\sqrt{(\Delta x)^2 + (\Delta y)^2}}$$

$$\vec{a} = a \cdot \vec{r}_n$$

3.3 Vinkel

Noe som jeg tenkte kunne være nyttig i programmet var å skaffe vinkelen som prosjektilen blir skutt i. Jeg valgte å velge å gi vinkelen mellom fartsvektoren og hva kraftsvektoren hadde vært i en sirkelbevegelse. I praksis lagde jeg en vektor fra himmellegemet til prosjektilet (motsatt retning av kraftvektoren

i en sirkelbevegelse) og sammenlignet det med vektoren som skal dannes mellom (x_1, y_1) og (x_2, y_2) som man ser i figur 4.



Figur 6: To vektorer dannes når man skyter prosjektilen

I figuren ovenfor er $\vec{u}$ vektoren fra himmellegemet til prosjektilen, mens $\vec{v}$ er vektoren som dannes av de to forskjellige museklikkene som bestemmer prosjektilens posisjon og fart. For å finne selve vinkelen bruker jeg litt vektorregning fra R1:

$$\vec{u} \cdot \vec{v} = |\vec{u}||\vec{v}| \cdot cos(\theta)$$

$$\theta = cos^{-1}\left(\frac{\vec{u} \cdot \vec{v}}{|\vec{u}||\vec{v}|}\right)$$

Merk at denne metoden ville ha gitt en vinkel på 90 grader i figur 5 som men ser ovenfor. Siden jeg synes at det er mer intuitiv å tenke på vinkelen i forhold til jordoverflaten tok jeg absoluttverdien av vinkelen jeg fikk, minus 90 grader.

4 Sjekke programmet og utforskning

4.1 Sjekke

Ved å bruke programmet mitt kan jeg sjekke om det funker i forhold til det matten tilsier. F.eks. så kan jeg sjekke om sirklingshastigheten stemmer. Det jeg gjør er å plassere prosjektilet så vidt over overflaten på himmellegemet. Deretter gir jeg den en fart som er lik eller så vidt høyere enn det som står i tabell 2. Det ser ut til å stemme for de få jeg sjekkeyt. På liknende måte er det mulig å sjekke resultatene i tabell 1.

4.2 Utforskning

Om man leker litt med programmet kan man kanskje legge merke til noe. F.eks. hvis man får et ikke-sirkulær bane, vil farten variere. Dette gir mening pga bevaring av mekanisk energi - når objekten er nærmere himmellegemet har den lavere potensiell energi som må kompenseres med en høyere kinetisk energi - og dermed en høyere fart.

At programmet viser banen før man har faktisk skutt prosjektilen gjør at man kanskje får en bedre forståelse for hvordan en endring i vinkelen påvirker banen man får. Den eneste måten man kan få en sirkelbane er ved å få en vinkel på 0 grader (mellom jordens overflate og fartsvektoren). Dette gir mening siden det er nødvendig å ha en 90 graders vinkel mellom fartsvektoren og kraftsvektoren slik at det skal være en sirkelbevegelse.

5 Feilkilder

Hvis man tester programmet litt nøye ser man at det gir nærme, men ikke alltid fullstendig riktig med de teoretiske verdiene. Dette kan det være en del grunner for.

For det første handler det muligvis om måten jeg har fått inn planetene. Jeg måtte få de inn som et bilde og fjerne bakgrunnen. Om jeg har gjort noe feil med bildeforholdene (det må ha lik høyde og bredde) eller størrelse så kan dette påvirke det visuelle i hvert fall.

En annen mulig feil man kan påpeke er valg av dt. Generelt kan man si at lavere verdi av dt gir bedre resultater. I mitt tilfelle valgte jeg å bruke samme verdi av dt på 10 sekunder som boken gjorde i det ene eksempelet jeg brukte. Jeg antar det funker siden boken brukte det, men er usikker hvor mye feil som faktisk skjer om man lar simuleringstiden være flere år eller bare generelt i en litt lengre periode enn det jeg testet med.

6 Konklusjon

I dette prosjektet har jeg undersøkt hvordan gravitasjonskraft påvirker objekter i bane rundt en himmellegeme, både teoretisk og gjennom simulering i Python og Pygame. Jeg har beregnet unnslipningsfart og banehastighet, og programmet mitt har vist hvordan ulike banetyper oppstår. Simuleringen bekreftet i stor grad de teoretiske formlene og ga en visuell forståelse av

bevegelsene.

Om jeg skulle ha jobbet videre med dette prosjektet kunne det vært aktuelt å blant annet:

- Fikse feilkilder så mye som mulig
- Utforske elliptiske baner mer som kanskje er litt mer spennende
- Lagt til flere himmellegemer og se hvordan objektene oppfører seg - programmering egner seg kanskje ekstra bra for dette.

7 Python kode

```
import pygame, sys, random, math
import numpy as np
from pygame.locals import *
pygame.init()


BACKGROUND = (255, 255, 255)

FPS = 60
fpsClock = pygame.time.Clock()
WINDOW_WIDTH = 600
WINDOW_HEIGHT = 600
bg = pygame.image.load("spaceBG.jpg")

dt =10
screen = pygame.display.set_mode((WINDOW_WIDTH, WINDOW_HEIGHT))
pygame.display.set_caption('My Game!')
G = 6.67430e-11

WHITE = (255, 255, 255)
GREEN = (0,128,0)
fallen = False
planetData = {
    "Earth": {
        "mass": 5.972e24,
        "radius": 6.371e6,
        "color": (0, 128, 0),
        "image": "theEarth.png"

    },
```

```
    "Venus": {
        "mass": 4.8675e24,
        "radius": 6.0518e6,
        "color": (255, 204, 153),
        "image": "venus.png"
    },
    "Mercury": {
        "mass": 3.3011e23,
        "radius": 2.4397e6,
        "color": (169, 169, 169),
        "image": "mercury.png"
    },
    "Mars": {
        "mass": 6.4171e23,
        "radius": 3.3895e6,
        "color": (255, 51, 51),
        "image": "mars.png"
    },
    "Jupiter": {
        "mass": 1.8982e27,
        "radius": 6.9911e7,
        "color": (255, 165, 0),
        "image": "jupiter.png"
    },
    "Saturn": {
        "mass": 5.6834e26,
        "radius": 5.8232e7,
        "color": (255, 223, 128),
        "image": "saturn.png"
    },
    "Uranus": {
        "mass": 8.6810e25,
        "radius": 2.5362e7,
        "color": (173, 216, 230),
        "image": "uranus.png"
    },
    "Neptune": {
        "mass": 1.02413e26,
        "radius": 2.4622e7,
        "color": (0, 0, 255),
        "image": "neptune.png"
    }
}
```

```
76 class Planet:
77     def __init__(self, x, y, mass, radius, color, image):
78         #recieve input in terms of pixels and converts to meters
79         self.x = x*scaleFactor
80         self.y = y*scaleFactor
81         self.mass = mass
82         self.radius = radius*scaleFactor
83         self.color = color
84         self.imagePath = image
85         self.img = pygame.image.load(self.imagePath)
86         self.radius_scaled = int(self.radius / scaleFactor)
87         self.img = pygame.transform.scale(self.img, (self.
    radius_scaled * 2, self.radius_scaled * 2))
88 
89 
90 
91     def draw(self):
92         self.img = pygame.image.load(self.imagePath)
93         self.radius_scaled = int(self.radius / scaleFactor)
94         self.img = pygame.transform.scale(self.img, (self.
    radius_scaled * 2, self.radius_scaled * 2))
95         pygame.draw.circle(screen, self.color, (int(self.x/
    scaleFactor), int(self.y/scaleFactor)), self.radius/
    scaleFactor)
96         screen.blit(self.img, (int((self.x-self.radius)/
    scaleFactor), int((self.y-self.radius)/scaleFactor)))
97 
98 
99 
100 class Projectile:
101     def __init__(self, x, y, mass, radius, vx, vy):
102         self.x = x*scaleFactor
103         self.y = y*scaleFactor
104         self.pos = [self.x,self.y]
105         self.mass = mass
106         self.radius = radius #set to 1e6 large size just for
    visuals
107         self.vx = vx*scaleFactor
108         self.vy = vy*scaleFactor
109         self.exist = True
110         self.startTime = pygame.time.get_ticks() / 1000.0
111         self.time = 0
112         self.positions = []
113         self.timesUpdated = 0
114 
```

```
115
116     def update(self, planet, realTime, fallen):
117         self.x += self.vx*dt
118         self.y += self.vy *dt
119         self.pos = [self.x, self.y]
120         self.time = pygame.time.get_ticks() / 1000.0 - self.
    startTime
121         self.fallen = fallen
122         self.timesUpdated+=1
123
124
125         dx = self.x - planet.x
126         dy = self.y - planet.y
127         r = math.sqrt(dx**2 + dy**2)
128
129         F = (G * self.mass * planet.mass) / r**2
130         a = F / self.mass
131
132         dirVec = np.array((planet.x - self.x, planet.y - self.y)
    )
133         eV = dirVec / np.linalg.norm(dirVec)
134
135         aVector = a * eV
136         self.vx += aVector[0] * dt
137         self.vy += aVector[1] * dt
138         #print(math.sqrt(self.vx**2+self.vy**2)*1e-3)
139         pygame.draw.circle(screen, WHITE, (int(self.x /
    scaleFactor), int(self.y / scaleFactor)), 10)
140
141         dist = r
142         if dist < planet.radius:  # does not take into account
    the object's radius
143             self.exist = False
144             self.fallen = True
145             print("jaja")
146         if self.timesUpdated%200==0:
147             self.positions.append([self.x,self.y])
148
149     def draw(self):
150         if len(self.positions) > 1:
151             for i in range(1, len(self.positions),3):
152                 start_pos = (int(self.positions[i-1][0] /
    scaleFactor), int(self.positions[i-1][1] / scaleFactor))
153                 end_pos = (int(self.positions[i][0] /
    scaleFactor), int(self.positions[i][1] / scaleFactor))
```

```
154                 #pygame.draw.line(screen, WHITE, start_pos,
    end_pos, 4)
155         maxTrailLength = 10
156         if len(self.positions) > maxTrailLength:
157             self.positions.pop(0)
158 
159     def drawPath(self, planet, steps=100, timeStep=dt):
160         predictedPositions = []
161         xPred = self.x
162         yPred = self.y
163         vxPred = self.vx
164         vyPred = self.vy
165 
166         for _ in range(steps):
167             dx = xPred - planet.x
168             dy = yPred - planet.y
169             r = math.sqrt(dx**2 + dy**2)
170 
171             F = (G * self.mass * planet.mass) / r**2
172             a = F / self.mass
173 
174             dirVec = np.array((planet.x - xPred, planet.y -
    yPred))
175             eV = dirVec / np.linalg.norm(dirVec)
176 
177             aVector = a * eV
178             vxPred += aVector[0] * timeStep
179             vyPred += aVector[1] * timeStep
180 
181             xPred += vxPred * timeStep
182             yPred += vyPred * timeStep
183 
184             predictedPositions.append([xPred, yPred])
185 
186         return predictedPositions
187 
188 
189 
190 def firstClick(num):
191     if num%2!=0:
192         return True
193 
194 def main ():
195     global scaleFactor
196     scaleFactor = 1e5
```

```
197     global projectile
198     projectileRadius = 1e6
199     looping = True
200     clicks = 0
201     projectiles = []
202 
203     simulationTime = 0.0
204     realTime = pygame.time.get_ticks() / 1000.0
205     simulationSpeed = 10
206     font = pygame.font.SysFont('Arial', 16)
207     dx = 0
208     dy = 0
209     x1Pos = 0
210     y1Pos = 0
211     moving = False
212     zoom = 1.2
213 
214     planetChoices = list(planetData.keys()) #returns a list of
        all names of the planets - nested dictionary is why the key
        in this case returns the names and not the other values.
215     planetIndex = 0
216     planetChoice = "Earth"
217     info = planetData[planetChoice]
218     planet = Planet(300, 300, info["mass"], float(info["radius"
        ]/scaleFactor), info["color"], info["image"])
219 
220     while looping:
221         for event in pygame.event.get() :
222           if event.type == QUIT :
223             pygame.quit()
224             sys.exit()
225           if event.type == KEYDOWN:
226               for projectile in projectiles:
227                   projectile.dx = planet.x-projectile.x
228                   projectile.dy = planet.y-projectile.y
229               if event.key == pygame.K_RIGHT:
230                   if planetIndex <(len(planetChoices)-1):
231                       planetIndex+=1
232                       planetChoice = planetChoices[planetIndex]
233                       info = planetData[planetChoice]
234                       planet = Planet(300, 300, info["mass"],
        float(info["radius"]/scaleFactor), info["color"], info["image
        "])
235                       print(planetChoice)
236 
```

```
237             elif event.key == pygame.K_LEFT:
238                         if planetIndex>0:
239                             planetIndex-=1
240                             planetChoice = planetChoices[
    planetIndex]
241                             info = planetData[planetChoice]
242                             planet = Planet(300, 300, info["
    mass"], float(info["radius"]/scaleFactor), info["color"],
    info["image"])
243                             print(planetChoice)
244             elif event.key == pygame.K_d:
245                 projectiles=[]
246
247
248             if event.key == K_PLUS or event.key == K_EQUALS:
249                 scaleFactor /= zoom
250                 planet.radius_scaled = int(planet.radius /
    scaleFactor)
251                 planet.img = pygame.transform.scale(planet.img
    , (planet.radius_scaled * 2, planet.radius_scaled * 2))
252                 planet.x /= zoom
253                 planet.y /= zoom
254
255                 #using the fact that the distance in meters
    between the planet and the projectiles are constant
    idependent of the zoom. Use minus since pygame uses different
     coordinate system(??)
256                 for projectile in projectiles:
257                     projectile.x = planet.x-projectile.dx
258                     projectile.y = planet.y-projectile.dy
259
260
261             elif event.key == K_MINUS:
262                 scaleFactor *= zoom
263                 planet.radius_scaled = int(planet.radius /
    scaleFactor)
264                 planet.img = pygame.transform.scale(planet.img
    , (planet.radius_scaled * 2, planet.radius_scaled * 2))
265                 planet.x *= zoom
266                 planet.y *= zoom
267
268                 for projectile in projectiles:
269                     projectile.x = planet.x-projectile.dx
270                     projectile.y = planet.y-projectile.dy
271
```

```
272
273         if event.type == pygame.MOUSEBUTTONDOWN:
274             clicks+=1
275             pos = pygame.mouse.get_pos()
276             if firstClick(clicks):
277                 x1Pos = pos[0]
278                 y1Pos = pos[1]
279
280             else:
281                 simulationTime = 0
282                 x2Pos = pos[0]
283                 y2Pos = pos[1]
284                 dx = x2Pos-x1Pos
285                 dy = y2Pos-y1Pos
286
287                 projectiles.append(Projectile(x1Pos, y1Pos,
    1000, 1e6, dx*1e-3, dy*1e-3))
288                 print(round(math.sqrt((dx*1e-3*scaleFactor)
    **2+(dy*1e-3*scaleFactor)**2)*1e-3, 2), "km/s")
289
290                 dx = x1Pos * scaleFactor - planet.x
291                 dy = y1Pos * scaleFactor - planet.y
292                 distance_m = math.sqrt(dx**2 + dy**2)
293                 surface_distance_m = distance_m - planet.
    radius #does not take into account the radius of the
    projectile - size of the projectiles is only for visuals
294                 print(int(surface_distance_m * 1e-3), "km unna
     (fra overflate)")
295
296     if projectiles:
297         timeElapsed = projectiles[-1].time
298     else:
299         timeElapsed = 0.0
300     screen.blit(bg, (0, 0))
301
302
303     userPos = pygame.mouse.get_pos()
304
305     if firstClick(clicks):
306         pygame.draw.line(screen, GREEN, (x1Pos, y1Pos), (
    userPos[0], userPos[1]), 4)
307         pygame.draw.circle(screen, WHITE, (x1Pos, y1Pos),
    10) #1e6/scaleFactor
308
309     for i in range(simulationSpeed):
```

```
310             for projectile in projectiles:
311                 if projectile.exist:
312                     projectile.update(planet, realTime, fallen)
313                     projectile.draw()
314             #updates simulationTime only if the last projectile
    - the one of interest - has not fallen yet
315             if projectiles and projectiles[-1].fallen == False:
316                 #simulationTime += dt * simulationSpeed / FPS
317                 realTime = pygame.time.get_ticks() / 1000.0
318                 #simulationTime = projectiles[-1].time*100
319                 simulationTime += dt
320
321
322
323         planet.draw()
324         userPos = pygame.mouse.get_pos()
325         mouseX = userPos[0]
326         mouseY = userPos[1]
327         dx = mouseX-x1Pos
328         dy = mouseY-y1Pos
329         speed = math.sqrt(dx**2+dy**2)*1e-1
330         speed = round(math.sqrt((dx*1e-3*scaleFactor)**2+(dy*1e
    -3*scaleFactor)**2)*1e-3, 2)
331
332         if clicks%2==1:
333             tempProjectile = Projectile(x1Pos, y1Pos, 1000, 1e6,
     (userPos[0] - x1Pos) * 1e-3, (userPos[1] - y1Pos) * 1e-3)
334
335             predictedPath = tempProjectile.drawPath(planet,
    steps=5000)
336             for i in range(1, len(predictedPath)):
337                 start_pos = (int(predictedPath[i-1][0] /
    scaleFactor), int(predictedPath[i-1][1] / scaleFactor))
338                 end_pos = (int(predictedPath[i][0] / scaleFactor
    ), int(predictedPath[i][1] / scaleFactor))
339                 pygame.draw.line(screen, WHITE, start_pos,
    end_pos, 2)
340
341
342         if firstClick(clicks):
343             mousePlanetDist = math.sqrt((x1Pos * scaleFactor -
    planet.x)**2 + (y1Pos * scaleFactor - planet.y)**2) - planet.
    radius
344         else:
345             mousePlanetDist = math.sqrt((mouseX * scaleFactor -
```

```
    planet.x)**2 + (mouseY * scaleFactor - planet.y)**2) - planet
    .radius
346
347
348         mousePlanetDist = round(mousePlanetDist*1e-3, 2)
349
350         if projectiles and projectiles[-1].exist: #Only works if
     there is a projectile
351             dxBallPlanet = projectiles[-1].x-planet.x
352             dyBallPlanet = projectiles[-1].y-planet.y
353         else:
354             dxBallPlanet = 0
355             dyBallPlanet = 0
356         ballPlanetVec = np.array((dxBallPlanet, dyBallPlanet))
357         ballPlanetDist = (np.linalg.norm(ballPlanetVec)-planet.
    radius)*1e-3 #Distance between the ball and the planet at all
     times
358
359         dxInitBallPlanet = x1Pos*scaleFactor-planet.x
360         dyInitBallPlanet = y1Pos*scaleFactor-planet.y
361         initballPlanetVec = np.array((dxInitBallPlanet,
    dyInitBallPlanet)) #Vector from the initial ball position to
    the planet (earth)
362
363         dxBallMouse = mouseX*scaleFactor-x1Pos*scaleFactor
364         dyBallMouse = mouseY*scaleFactor-y1Pos*scaleFactor
365         ballMouseVec = np.array((dxBallMouse, dyBallMouse)) #
    Vector from the initial ball position to the mouse position
366
367         #theta = math.acos((np.dot(initballPlanetVec,
    ballMouseVec))/(np.linalg.norm(initballPlanetVec)*np.linalg.
    norm(ballMouseVec)))
368         theta = math.acos(np.clip(np.dot(initballPlanetVec,
    ballMouseVec) / (np.linalg.norm(initballPlanetVec) * np.
    linalg.norm(ballMouseVec)), -1.0, 1.0))
369         #TODO research np.clip
370         theta = abs(np.rad2deg(theta)-90)
371         #print(theta)
372
373
374
375
376         timeText = font.render(f"Sim time: {simulationTime:.1f}s
     | Real time: {timeElapsed:.1f}s | Distance from planet {
    ballPlanetDist:.1f}", True, WHITE)
```

```
377         simText = font.render(f"New ball distance from surface {
    mousePlanetDist:.0f}km | Speed {speed:.1f}km/s | Angle {theta
    :.0f}  ", True, WHITE)
378 
379         #if not projectiles:
380          #   projectileInfo = font.render("none", True, WHITE)
381         #else:
382          #   if projectiles[-1].fallen:
383         projectileInfo = font.render(f"Planet: {planetChoice}",
    True, WHITE)
384 
385         screen.blit(timeText, (10, 10))
386         screen.blit(simText, (10, 30))
387 
388         screen.blit(projectileInfo, (10,50))
389 
390         pygame.display.update()
391 
392         fpsClock.tick(FPS)
393 
394 main()
```

8 Kilder

- https://snl.no/ellipse_-_matematikk
- https://www.youtube.com/watch?v=HTfwhmHVpqM&t=725s
- https://www.mn.uio.no/astro/tjenester/publikum/ol-astronomi-og-astrofysikk/ressurser/orbital_mechanics.pdf
- Ergo Fysikk 2